Comprehensive Analysis of Q-Learning, SARSA and Expected SARSA

On a High-Penalty Grid World (Fox 2016)

Part III

Team: Reward Hunters

Abstract

In this project we compare three well-known reinforcement learning algorithms—Q-Learning, SARSA, and Expected SARSA—on an 8×8 grid world with obstacles and heavy penalties for wrong moves. The environment is from Fox (2016). Expected SARSA reaches the lowest total cost and has the smallest variation between runs. SARSA is also safe but a little less efficient, while Q-Learning sometimes makes risky decisions and ends up with higher cost. We also tested different learning rates (α) and exploration rates (ε). A small α (about 0.1–0.3) and ε around 0.1 give the best performance.

1 Introduction

Reinforcement learning teaches an agent to act in an environment by trial and error. Three popular methods are Q-Learning, SARSA, and Expected SARSA. They all update a table of Q-values according to different rules, and this can lead to different behaviour when the environment has random events and big penalties.

In this work we implement all three and test them on a grid world where the agent must reach a goal while avoiding blocked cells. If the agent tries to enter a blocked cell or move outside the grid, it gets a huge penalty of +1000. The goal is to see which algorithm learns the safest and cheapest path.

2 Environment and Algorithms

2.1 The Grid World

- An 8×8 grid (64 cells). The goal is cell [4,4] (index 36).
- 15 cells are blocked and cannot be entered.
- The agent can take 9 actions: the 8 compass directions and a "stay" action.
- After every action there is a 30% chance that the agent drifts to a neighbouring cell.
- Cost: +1 for every step, plus a small Gaussian noise ($\sigma = 0.2$). If the move is invalid (blocked or off-grid) an extra +1000 is added.
- The episode ends when the goal is reached.

2.2 The Algorithms

All three algorithms use an ε-greedy policy: with probability ε the agent picks a random action, otherwise it picks the action with the highest Q-value. The main difference is in the update

formula.

Q-Learning (off-policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

SARSA (on-policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big]$$

Expected SARSA (off-policy, uses expectation)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \sum_{a'} \pi(a'|s') Q(s',a') - Q(s,a)\big]$$

Here α is the learning rate and γ is the discount factor.

3 Experimental Setup

We wrote the code in Python using NumPy and Matplotlib. Each algorithm was run for 100 episodes, and we repeated every experiment 5–10 times (with different random seeds) to get reliable averages. The default parameters were $\alpha = 0.5$, $\varepsilon = 0.1$, $\gamma = 0.9$ for the first comparison, but later we varied these to see how sensitive the algorithms are.

Performance is measured by the **total cost in one episode** – lower is better. We plot learning curves (cost vs. episode), heatmaps of visited states, and cost as function of different parameters.

4 Results and Discussion

4.1 Learning Curves

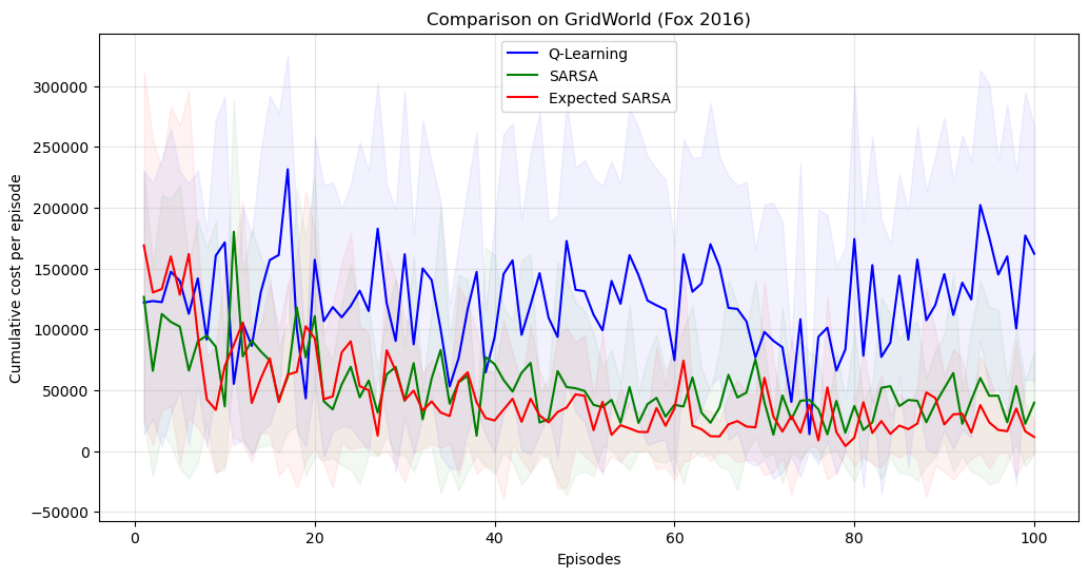


Figure 1: Average total cost per episode over 10 runs. The shaded bands show ±1 standard deviation. Expected SARSA (red) learns the fastest and stays the most stable.

From Figure 1 we see that all three methods improve. **Expected SARSA** (red) achieves the lowest cost and the narrowest shaded band, meaning it is the most reliable. **SARSA** (green)

works almost as well but has a bit more fluctuation. **Q-Learning** (blue) has the highest cost and very wide spread – it sometimes makes overly optimistic choices that cause a heavy penalty when things go wrong.

4.2 Where Does the Agent Go? (Heatmaps)

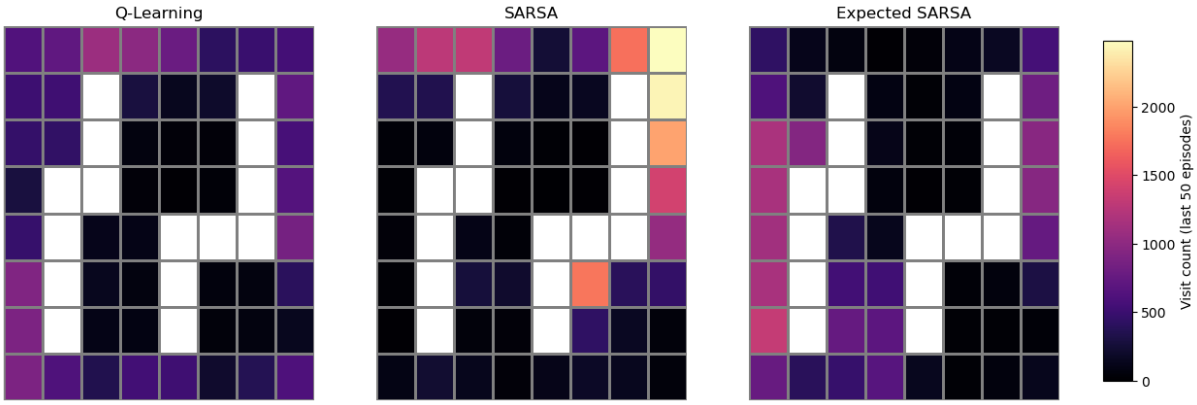


Figure 2: How often each state was visited in the last 50 episodes. Darker cells were visited more. White cells are blocked.

The heatmaps (Figure 2) show the learned paths after training. Q-Learning wanders around and sometimes gets close to blocked cells. SARSA follows a very narrow, safe path and barely visits other states. Expected SARSA takes a balanced route – it stays safe but does not get stuck in a single narrow corridor like SARSA.

4.3 Changing the Learning Rate (α)

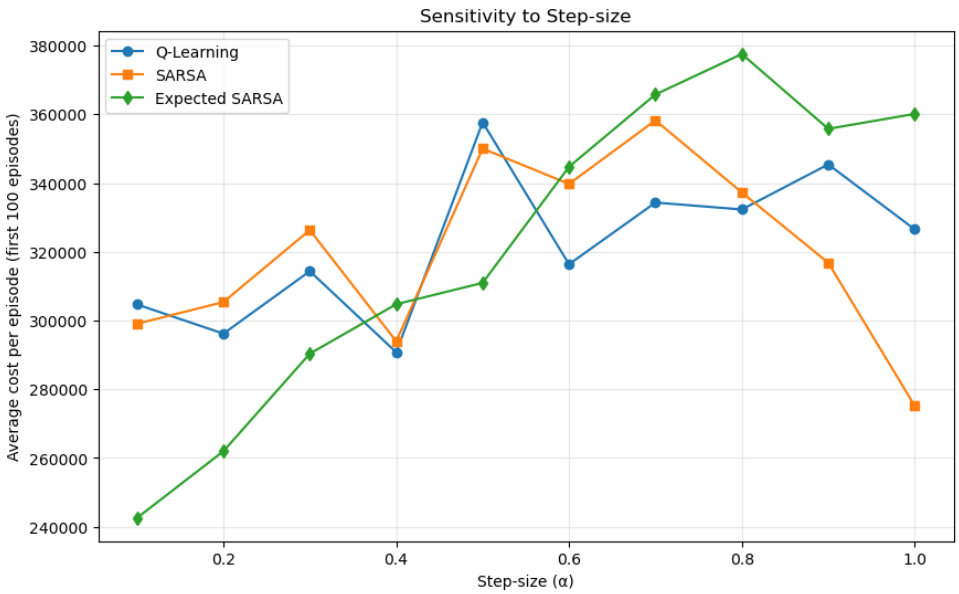
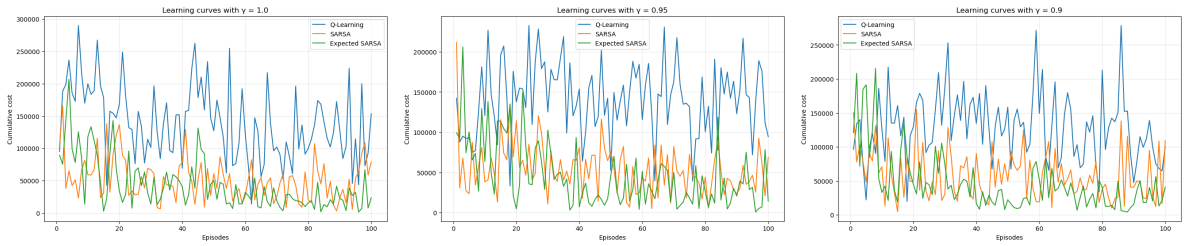


Figure 3: Average cost over the first 100 episodes for different α. Expected SARSA works best with small α, while Q-Learning and SARSA are unpredictable.

Figure 3 shows that Expected SARSA's cost goes up as α increases. It does best at small values like 0.1 or 0.3. Q-Learning and SARSA jump around a lot – they are sensitive and one cannot

rely on a single α value. The lesson is: keep α small (about 0.1–0.3) for stable learning.

4.4 Effect of Discount Factor (γ)



(a) $\gamma = 1.0$ (b) $\gamma = 0.95$ (c) $\gamma = 0.9$

Figure 4: Learning curves for three different γ values. The ranking Expected SARSA > SARSA > Q-Learning stays the same.

When we reduce γ (Figure 4), the curves become a little flatter, but Expected SARSA is still the best. So our findings are not tied to one particular choice of γ.

4.5 Exploration Rate (ε)

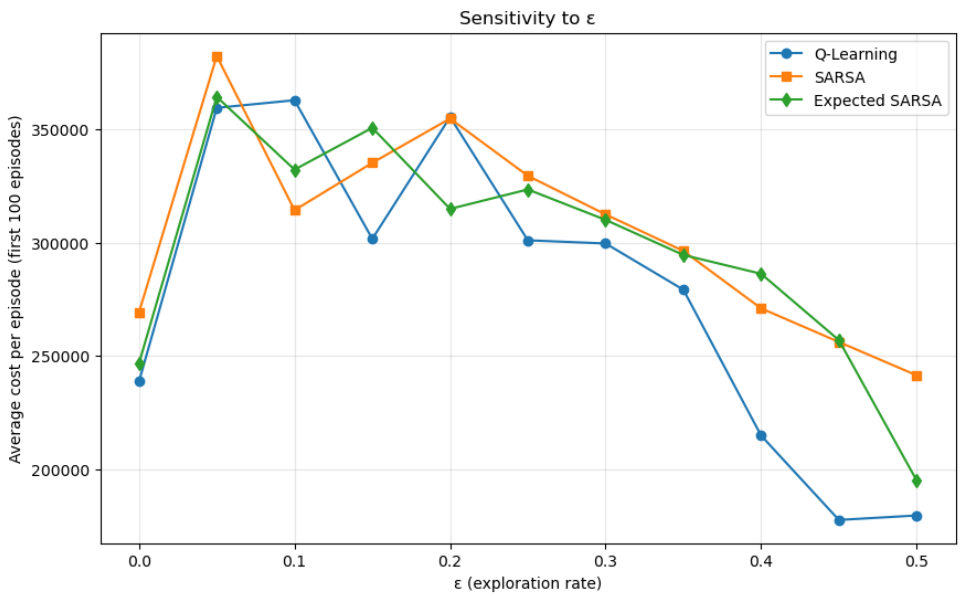
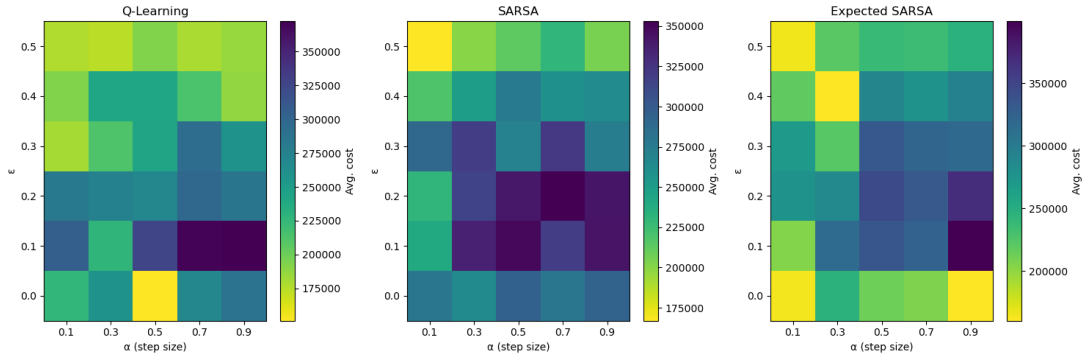


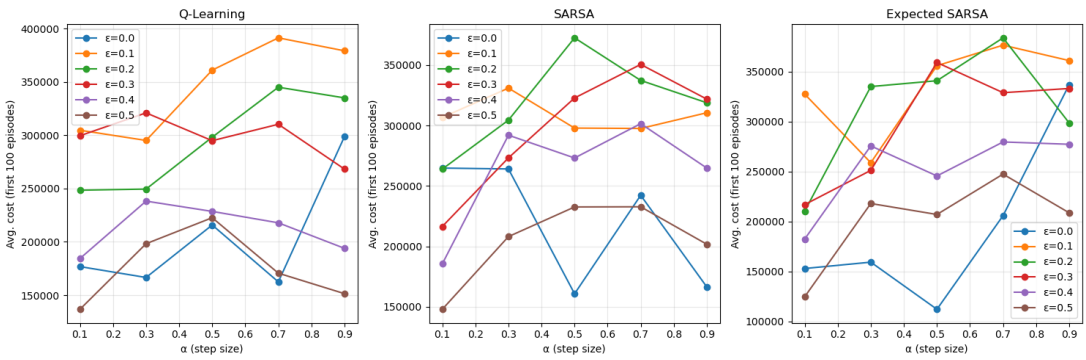
Figure 5: Cost vs. ε. Intermediate ε (0.05–0.3) does the most damage. Very high ε reduces cost because the agent falls into a penalty early and the episode ends quickly.

Figure 5 gives a surprising result. Adding just a little exploration ($\varepsilon \approx 0.05$) makes the cost jump up sharply, because once in a while the agent hits a blocked cell and receives +1000. As ε becomes very large (above 0.3), the cost drops again – but this is not because the agent becomes smarter; it simply crashes early and the episode stops before many normal steps accumulate. For good learning, ε around 0.1 is the best.

4.6 Putting ε and α Together



(a) Heatmaps: lighter = lower cost.



(b) Line view: cost vs. α for each ε.

Figure 6: Combined effect of ε and α. ε is much more important than α in this test range.

The grid plots (Figure 6) confirm that the exploration rate ε dominates. The lowest cost appears at $\varepsilon = 0.0$ or 0.1 together with a small α (0.1–0.3). For a fixed ε, changing α has only a small effect.

5 Conclusion

After running many experiments we can say:

- **Expected SARSA** is the safest and most consistent algorithm on this grid world. It gives the smallest average cost and the least variation.
- **SARSA** is also safe but can be a little less efficient.
- **Q-Learning** often makes over-optimistic decisions that lead to big penalties and high cost.
- The best hyper-parameters are a small learning rate $\alpha \approx 0.1$–0.3 and an exploration rate ε around 0.1.
- Too much exploration ($\varepsilon > 0.3$) makes the agent fail early and hides the real performance; too little ($\varepsilon < 0.1$) can prevent it from finding a good path.

All the code and detailed results can be found in the Jupyter notebook that came with this report.